

DesignKata

LLD practice with feedback you can check.

Pick a low-level design problem, write short notes and a class diagram, and submit. Rules check the structure in about a second. An AI reviewer then judges six qualities, and every judgment has to quote your own words. Code verifies each quote. Then take the twist: a new requirement, the way an interviewer adds one. DesignKata diffs your two class diagrams and tells you how many existing classes the change forced you to edit.

Built by Devank Srivastava for the CipherSchools engineering assignment, September 2026.

The screenshot shows the DesignKata web application interface. At the top, there's a navigation bar with 'DesignKata', 'Problems', and 'Progress' links. A status bar indicates 'AI review on: deepseek/deepseek-v4.1-flash'. The main heading is 'Feedback on version 2' for the 'PARKING LOT · VERSION 2 · TWIST: EV CHARGING SPOTS' problem. A progress bar shows 'SUBMITTED' → 'EVALUATING' → 'COMPLETED'. A section titled 'WORK ON THIS NEXT' highlights 'Behaviour and edge cases' with a prompt to add a short flow for a lost-ticket case. Below this, a summary states the design is strong on responsibilities, coupling, and extensibility, but needs more explicit priorities and flows. A 'Rubric' section shows two criteria: 'Requirements and scope' (Level 3 of 4) and 'Responsibilities and cohesion' (Level 4 of 4), each with 'WHAT WORKS' and 'CONCERN' points.

Live demo: <https://designkata.onrender.com> (free host, first load after idle takes about a minute)

Documents: [Research note](#) · [Design note](#) · [Decision log](#) · [AI usage](#) · PDFs in [docs/pdf/](#)

Try it in 60 seconds

1. Open the [live demo](#), or run it locally with the two commands below. The free host sleeps when idle, so the first load can take about a minute.
2. Open **Parking Lot**, click **Load sample design**, then **Submit for review**. The sample is a deliberately typical first attempt.
3. Rule findings appear within about a second. The rubric review follows in about 15 seconds. Look for the green "Found in your design" quotes.
4. Click **Take the twist**, add an EV charging spot to the diagram, submit, and read the blast radius.
5. Open **Progress** to see the weaknesses that keep coming back.

Run it locally

Requires **Node 24** (see [.nvmrc](#)). Nothing else to install.

```
npm install
npm run dev
```

Open <http://localhost:3000>.

- **No database needed.** Without `MONGO_URI`, a local MongoDB starts by itself and keeps its data in `.data/mongo`, so your history survives restarts.
- **No key needed.** Without `OPENROUTER_API_KEY`, feedback is rules-only and the app says so. For the AI review, copy `.env.example` to `.env` and set `OPENROUTER_API_KEY`. The model defaults to `deepseek/deepseek-v4.1-flash`; set `LLM_MODEL` to use another one. `npm run check:llm` confirms the key works.

For production mode, run `npm run build && npm start`. Deployment is described in `render.yaml`: one web service, with MongoDB Atlas as the database.

How it works

The screenshot shows the DesignKata web application. At the top, there's a navigation bar with 'DesignKata', 'Problems', and 'Progress'. A status bar indicates 'AI review on: deepseek/deepseek-v4.1-flash'. The main content area is divided into two columns. The left column displays a problem titled 'Parking Lot' with a difficulty level of 'EASY' and a duration of '45 MINUTE PRACTICE'. The problem description states: 'A mall in Noida is replacing paper parking slips with an automated lot. Bikes, cars and trucks enter through several gates, get a ticket, park on one of several levels and pay at the exit. The operator wants to change pricing without calling a developer.' Below the description are 'Requirements' (6 numbered items) and 'Out of scope' (3 bullet points). The right column shows a 'Revising version 2.' message: 'Your draft starts from what you last submitted. Submitting creates version 3.' Below this are two tabs: 'Design notes' and 'Class diagram'. The 'Class diagram' tab is active, showing a 'Mermaid class diagram' with the following code:

```
classDiagram
    class ParkingLot {
        -List~Level~ levels
        +entryGate(int id) EntryGate
        +exitGate(int id) ExitGate
    }
    class EntryGate {
        -SpotAllocator allocator
        +admit(Vehicle v) Ticket
    }
    class ExitGate {
        -PricingStrategy pricing
        -PaymentMethod payment
        +release(Ticket t) Receipt
    }
    ParkingLot --> EntryGate
    ParkingLot --> ExitGate
    EntryGate --> Ticket
    ExitGate --> Receipt
```

 Below the code is a visual representation of the class diagram with boxes for 'ParkingLot', 'EntryGate', 'ExitGate', 'Ticket', and 'Receipt' connected by arrows.

Choose a problem → design (notes + Mermaid diagram) → submit (202, stored first)
→ rules (about 1 s) → AI rubric review, every quote verified (about 15 s)
→ revise (level changes per criterion) or take the twist (blast radius) → progress

A submission is five guided notes (requirements, entities, flows, trade-offs, edge cases) plus a class diagram written as Mermaid text. Because the diagram is text, it can be parsed into classes and relationships, checked, and diffed between versions.

Ten deterministic rules run instantly and cost nothing. They check the required sections, the diagram syntax, and whether the core concepts appear (matched by the names learners actually use). They also flag a god class, a part of the problem that varies but has no abstraction, dependency loops, orphan classes, public state, deep inheritance, and edge cases the notes never mention.

The rubric has six criteria, each on four levels that are described in words, and no total score. The AI reviewer must quote evidence before it picks a level, and `EvidenceVerifier` checks every quote against what you wrote.

When a review is slow or fails, nothing is lost. The submission is stored first and a background worker claims the review with a lease, so rule findings show while the AI works. Provider errors are retried with backoff, and a failed review keeps its rule findings and can be retried. If the worker crashes, its job is picked up again, and the same `Idempotency-Key` never creates a duplicate.

The design note has the domain model, the state machine, the failure table and both change tests.

CONCERN

The design does not describe how the lost-ticket flow checks the plate or how the daily maximum is obtained, and the EV charging flow at exit is not detailed.

NEXT STEP

Add a short flow for the lost-ticket case showing who looks up the plate and who supplies the daily maximum, and describe how EV charging is settled at exit.

Found in your design:

Entry: EntryGate asks SpotAllocator for a spot; the allocator reserves it with a compare-and-set so two gates cannot take the same spot; a Ticket is issued.

Found in your design:

Exit: ExitGate asks PricingStrategy for the fee, charges it through PaymentMethod, and only then releases the spot, which notifies DisplayBoard.

Found in your design:

Lot full: SpotAllocator returns nothing and the gate refuses entry.

The trade-ons section does not weigh alternatives for allocation or payment, and does not discuss the cost of the observer choice or the atomic reservation approach.

NEXT STEP

Add one or two sentences weighing an alternative for allocation (for example, lowest level first versus nearest) and for payment, and note the trade-off of the atomic reservation approach.

Found in your design:

Strategy for pricing, allocation and payment, because each varies independently.

Found in your design:

Observer for the display board so spots do not know about screens.

Found in your design:

Rejected: vehicle subclasses, because behaviour does not differ by type yet; an enum is enough.

How your design absorbed the twist

Measured by diffing the two class diagrams

0

BLAST RADIUS

Additive: no existing class had to change.

Added: EVChargingSpot, KwhPricing

Structure checks

Deterministic, instant and free

No structural problems found.

Revise this design

See your progress

Measured results

What	Result
Calibration: weak, medium and strong designs, 3 reviews each (<code>npm run calibrate</code>)	Ordered correctly on 6 of 6 criteria; the widest spread was 1 level; 119 of 124 quotes verified
Consistency: 5 reviews of one design, in 2 configurations (<code>npm run experiment:consistency</code>)	5 of 6 criteria gave the same level every run; 112 of 115 quotes verified
Dogfood: DesignKata's own class diagram through its own rules (<code>npm run dogfood</code>)	23 classes, no findings

Scripts

Command	What it does
<code>npm run dev</code>	API, worker and React app on one port, with hot reload
<code>npm test</code>	171 tests: unit, integration and end to end, against a real MongoDB with a fake LLM
<code>npm run typecheck</code>	Type-checks the server and the web app
<code>npm run check:text</code>	Fails on em dashes, en dashes or anything that looks like an API key (runs in CI)
<code>npm run build / npm start</code>	Production build and server
<code>npm run check:llm</code>	Confirms the OpenRouter key and model work
<code>npm run calibrate</code>	Checks that the reviewer ranks designs of known quality correctly
<code>npm run experiment:consistency</code>	The research note's experiment
<code>npm run dogfood</code>	Runs DesignKata's own design through its rules
<code>npm run pdf</code>	Builds the submission PDFs with headless Chrome

Project layout

shared/	the contract both sides share: types, and the rubric
server/src/domain/	Attempt, Submission, Evaluation (state machine), DesignModel, Problem
server/src/analysis/	one reader per submission format; the change-impact diff
server/src/evaluation/	the Evaluator seam, rules, LLM rubric evaluator, evidence verifier, pipeline
server/src/llm/	LlmClient seam, OpenRouter client with retries, fake client, structured output
server/src/persistence/	MongoDB models and repositories (idempotency, compare-and-set, leases)
server/src/app/	PracticeService (use cases), EvaluationWorker, progress, rate limiter
server/src/http/	routes, validation and error mapping
server/src/problems/	the four problems, as data
server/test/	unit and integration tests
web/src/	React app: Problems, Workspace, Feedback, Progress
docs/	notes, decision log, diagrams, experiment output, PDFs

Limitations

- No accounts. The browser keeps a random learner id; a real product would use CipherSchools sign-in.
- The rules are heuristics. Thresholds such as eight methods for a god class are starting points to tune with real learners.
- The Mermaid parser covers the syntax learners write, not all of Mermaid.
- The calibration set is small (three designs on one problem), and the reviewer is strict, so level 4 is rare.
- Rate limiting (12 reviews per learner per hour) is per process. A shared store would take over with more than one instance.

Author

Devank Srivastava · [LinkedIn](#) · [GitHub](#)

Licensed under MIT.

AI usage

I used **Claude Code** as my coding assistant: I set the design and constraints, used it to write most of the implementation and first drafts of the docs, and reviewed the result against 171 tests in CI. Inside the product, **DeepSeek v4.1 Flash via OpenRouter** is the rubric reviewer.

The decisions that mattered most:

1. **Submission format.** I chose guided notes plus a Mermaid class diagram over a drawing canvas. Text can be parsed for deterministic checks and diffed between versions, which is what makes the blast radius possible.
2. **Research.** I skipped a poll and measured LLM feedback directly instead. At temperature 0 a bare "rate it 1 to 10" score was repeatable but never said what to fix, so I moved the thesis from "consistent feedback" to "specific, checkable feedback".
3. **Model.** I picked OpenRouter so the provider is one environment variable. Reasoning on and off gave the same levels, 15 s vs 41 s, so it is off by default.
4. **Trusting the reviewer.** The model must quote evidence before picking a level, and code verifies every quote against the learner's text. The verifier caught the model quoting our own rule findings as the learner's words; I tightened the prompt and kept the check.
5. **Concurrency bug.** The first worker version could report idle while still claiming a job, which showed up as a flaky test. I fixed the root cause (`drain()` now waits for a running pass) instead of loosening the test.

One rule of mine is enforced by CI: no em dashes anywhere (`npm run check:text`). It caught generated code twice.